IT-Project
Autonomously driving Remote Control Car

Bohnstedt Timo , Löhr Tim and Palpanis Ionannis

Georg Simon Ohm University of Applied Science

IT Project Presentation

January 24, 2020

TECHNISCHE HOCHSCHULE NÜRNBERG
GEORG SIMON OHM

Overview

1 Background

2 Literature Survey

3 Hardware

4 Software

5 Machine Learning

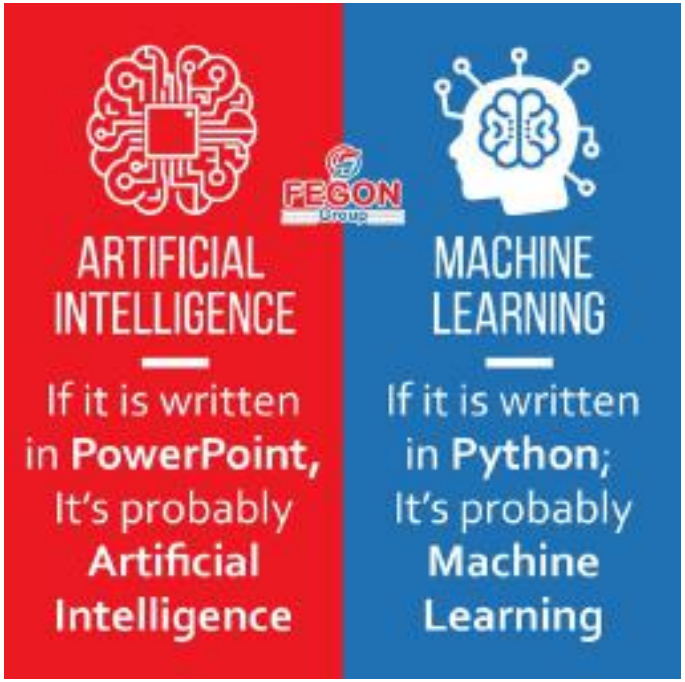
6 Autonomously Driving

7 Evaluation

Background

Background

"We can build these models, but we don't know how they work."
—Joel Dudley, MITs Technology Review



Literature Survey

The Level of autonomously driving Cars - Part I

Level 0

No Automation

- Driven by a human

Level 1

Driver Assistance

- Steering into the middle of the lane
- Breaking tires when the distance to the car in front is to close
- Just one system at a time

The Level of autonomously driving Cars - Part II

Level 2

Partial Automation

- A computer programm drives the car (accelerating and decelerating)
- The human is responsible and still in control

Level 3

Conditional Automation

- Like above but the computer looks over the environment as well
- The computer is able to handle easy standard scenarios
- Example: Google Waymo self-driving car

The Level of autonomously driving Cars - Part III

Level 4

High Automation

- The computer can handle every situation
- Without very special circumstances

Level 5

Conditional Automation

- The human is not required to be in the car
- E.g. it would be possible that cars search for parking lots by themselves

Reinforcement Learning - Part I

" [...] What we want is a machine that can learn form experience."

— Alan Turing, 1947

Reinforcement Learning - Part II

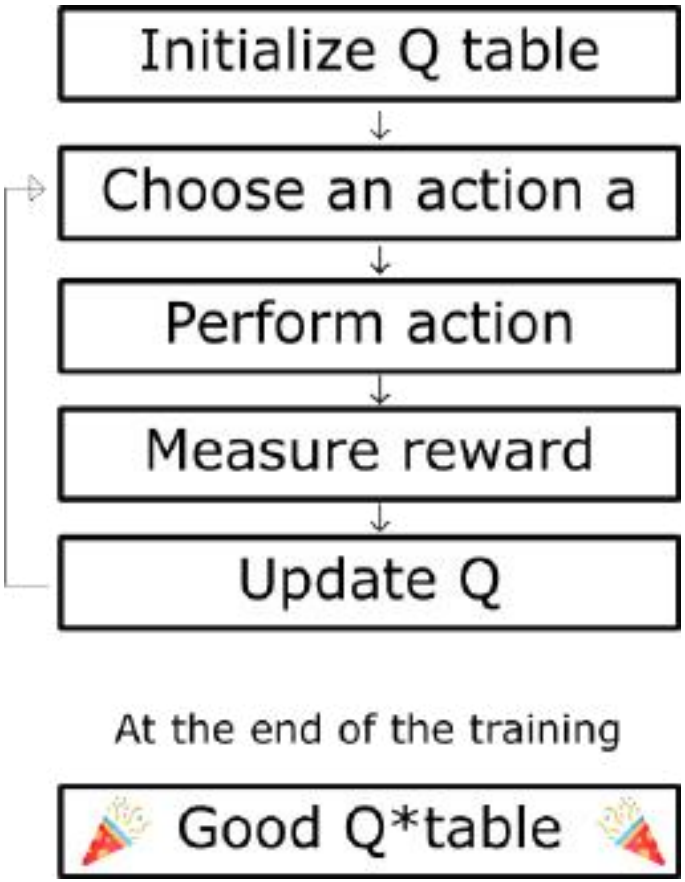
Advantages of unsupervised Learning

1. Can find every pattern (unknown objects)
2. Does not need examples, because it can learn from experience ...
3. ... so it is more useful for real-world problems (e. g. autonomous driving)

DDQN

Researchers from Google DeepMind (known from achievements at Boardgame GO) developed the Deep Reinforment Learning with Double Q-learning. They showed that reinforcement Learning can be implemented in (Donkey) Cars.

Reinforcement Learning - Part III



Reinforcement Learning - Part IV

"The basic idea is to get an Value Q which represents the state s and the action a of an unsupervised machine learning task"

$$\underbrace{NewQ(s,a)}_{\text{New Q value for that state and that action}} = \underbrace{Q(s,a)}_{\text{Current Q value}} + \underbrace{\alpha}_{\text{Learning Rate}}[\underbrace{R(s,a)}_{\text{Reward for taking that action at that state}} + \underbrace{\gamma}_{\text{Discount rate}} \underbrace{max\,Q'(s',a')}_{\text{Maximum expected future reward given the new s' and all possible actions at that new state}} - Q(s,a)]$$

Hardware

Remote Control Car

- Model car with a servomotor and electronic speed control (ESC)
- Weights only 1,88kg
- Easy to attach Raspberry Pi 3B+ and SunFounder Servodriver onto it

Raspberry Pi 3B+ - Part I

- Chosen over NVIDIA's Drive PX for this project, because the weight and power are suiting better
- 1.4GHz CPU to apply the Deep Learning to the captured images
- 16GB SanDisk SD-Card storage with Raspian Stretch Lite
- Powered through a Powerbank attached onto the RC-Car

Raspberry Pi 3B+ - Part II

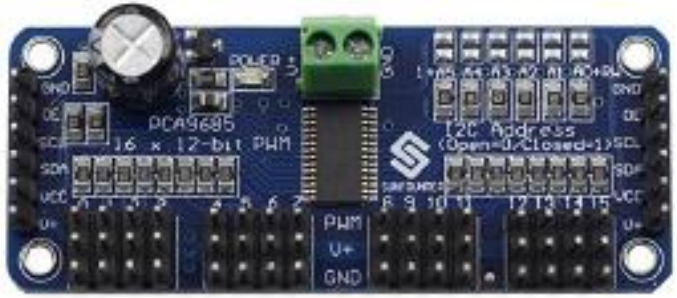
- For our purpose, we use the Rasp- berry Pi Camera Module 8MP v2.1. It can capture pictures with 1080p and 8-megapixel focus
- Connector on the Raspberry Pi 3B+ with a 15 Pin Ribbon Cable

PiCamera Module

Servodriver

From SunFounder, model *PCA9685*

- Cheap price: approximately 13
- Provides Pulse width modulation (PWM) to apply remote steering through the Raspberry Pi 3B+
- Powered with 3-5V from the Raspberry Pi 3B+



Cable Management - Part I

Raspberry Pi 3B+ to Servodriver

8 Pin's need to be connected.

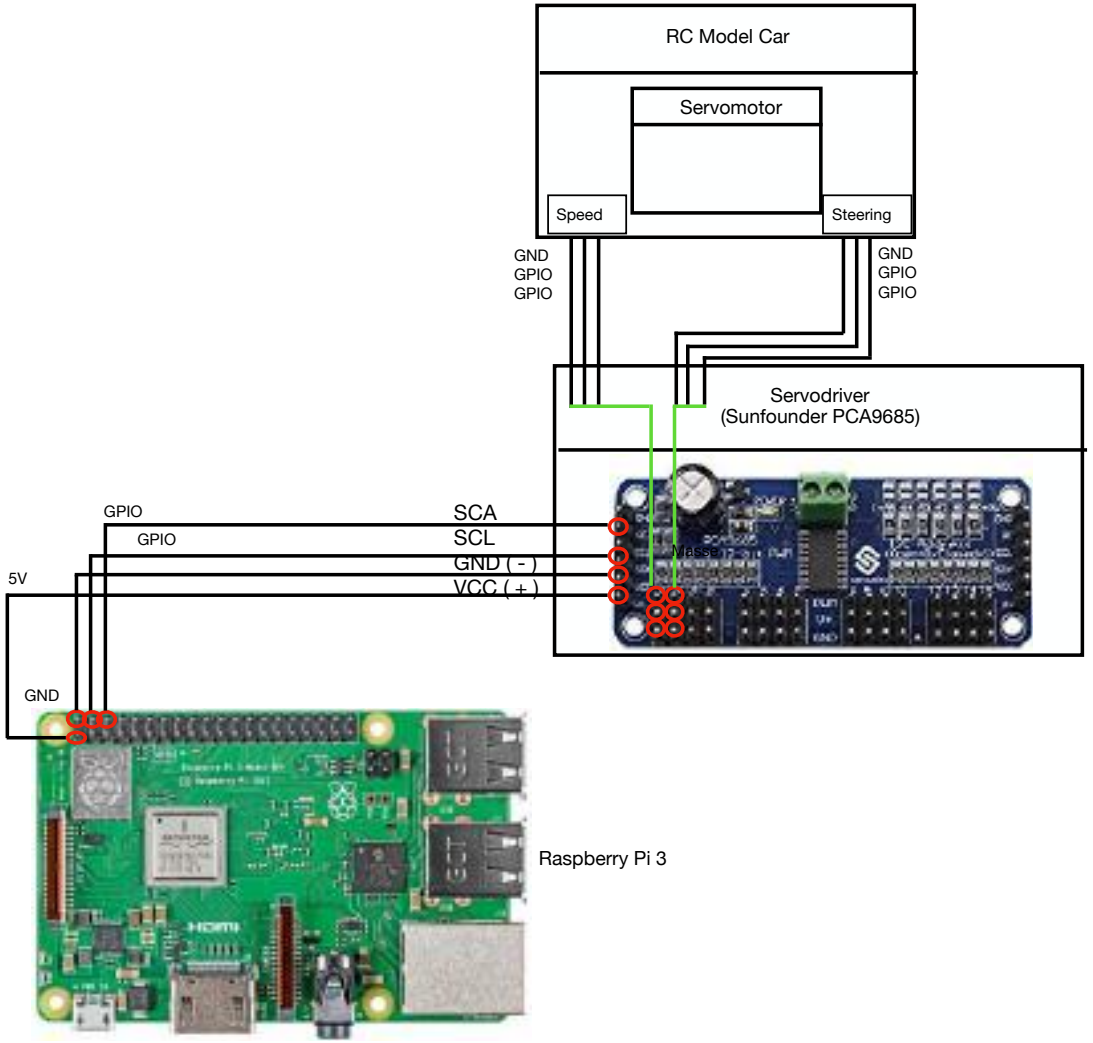
- Ground GND
- Power VCC 5 Volt
- 2x3 Pin's for the steering and throtteling

Servodriver to Servomotor

Four Pin's need to be connected.

- Ground GND
- Power VCC 5 Volt
- 2 Pin's for the steering and throtteling

Cable Management - Part II



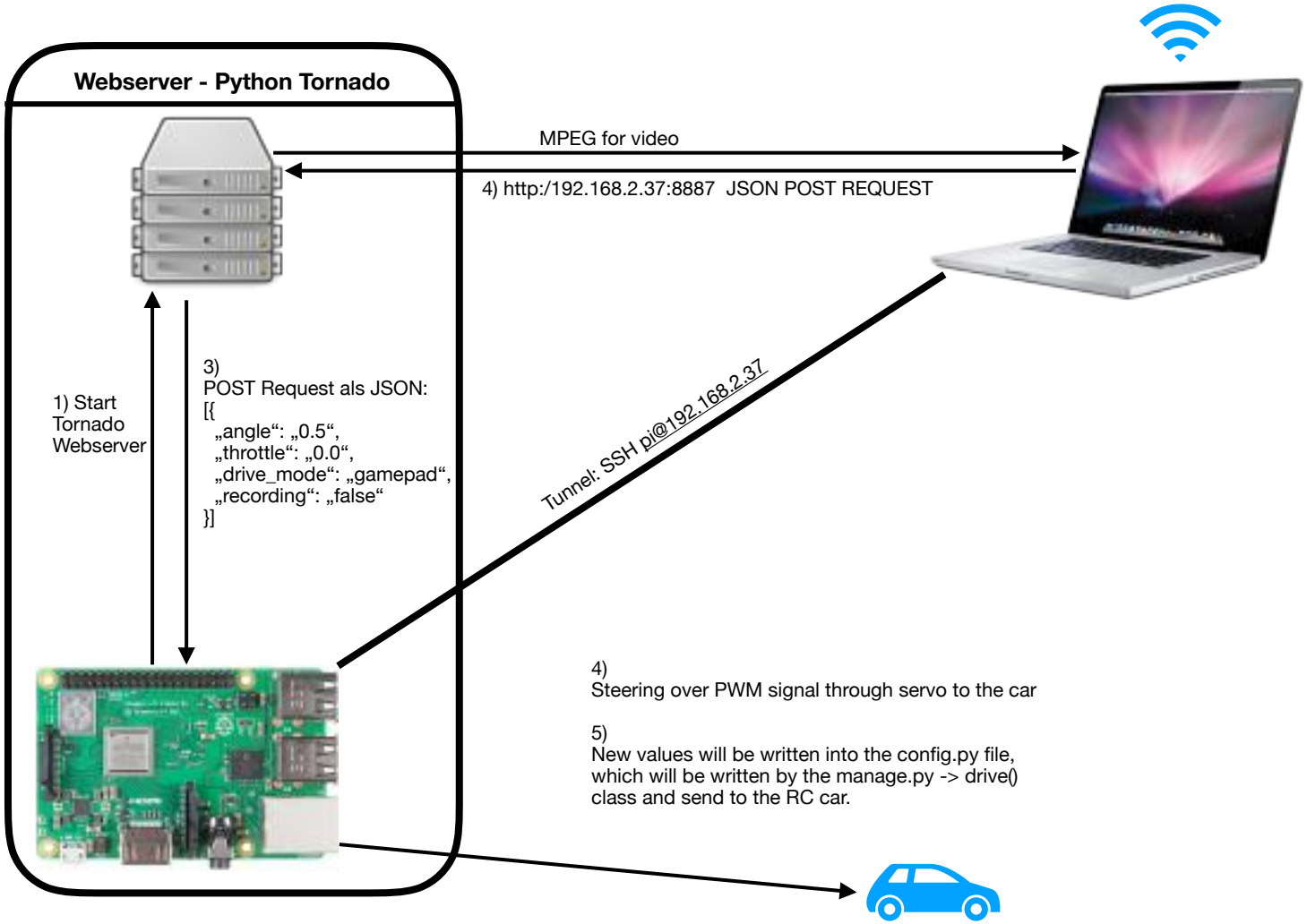
Software

Software Architecture - Python Files

The Donkey Self-Driving Car Project is the basis for our group Project. The Filesystem is:

- Management: Start the Webserver
- Parts: Controls the single parts, e.g. Camera, Steering, Throttle, ..
- Templates: Web Templates for the Webserver
- Test: Start the environment without error's

Python Tornado Webserver



Training the model with Unity

- For capturing enough images, someone from the DonkeyCar community programmed a Unity Game.
- Simulator of the real word with lines on the border of the track for the image recognition task

Snapshot from the Simulation

Machine Learning

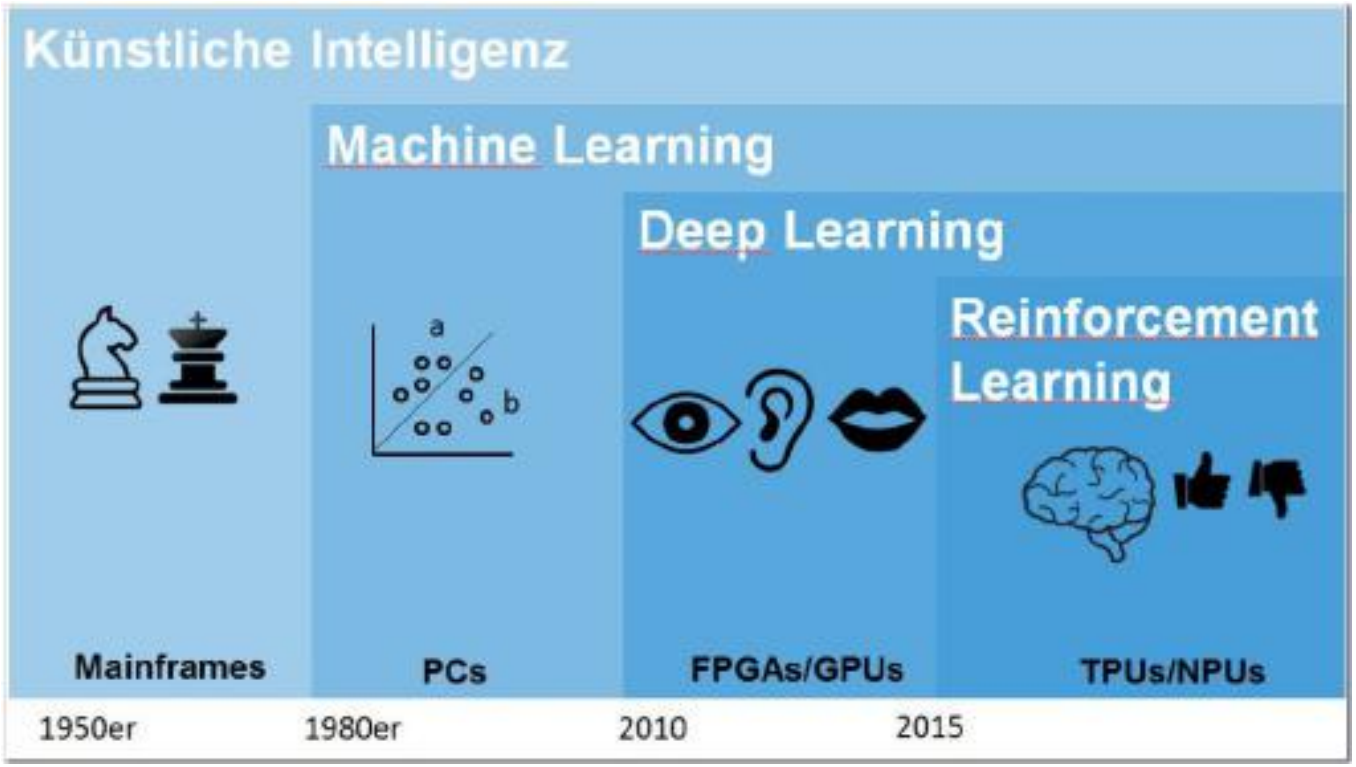
Machine Learning - Part I

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E"

— Mitchel , 1979

Machine Learning - Part II

- Casully explained: Machine Learning is a functions which could be optimized



Deep Learning - Part I

"We get a deep neural network if we had more than one hidden layer in a neural network . Neural Networks are a concept in the machine learning discipline which try to build a program that learns like the human brain is learning."

— Andrew Ng , 2013

Deep Learning - Part II

Parts of a Convolutional Neural Network:

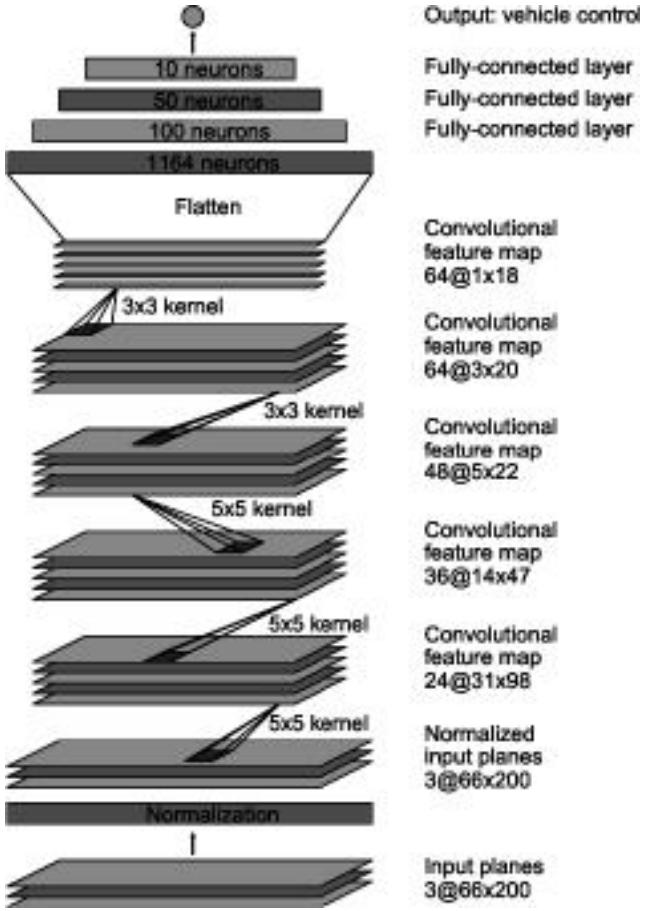
1. Convolutional Layers (CONV)
2. Pooling Layers (POOL)
3. Fully connected Layers (FC)

```
def default_linear():
    img_in = Input(shape=(120, 160, 3), name='img_in')
    x = img_in

    # Convolution2D class name is an alias for Conv2D
    x = Convolution2D(24, (5, 5), strides=(2, 2), activation='relu')(x)
    x = Convolution2D(32, (5, 5), strides=(2, 2), activation='relu')(x)
    x = Convolution2D(64, (5, 5), strides=(2, 2), activation='relu')(x)
    x = Convolution2D(64, (3, 3), strides=(2, 2), activation='relu')(x)
    x = Convolution2D(64, (3, 3), strides=(1, 1), activation='relu')(x)
    x = Flatten(name='flattened')(x)  # Flatten to 1D (Fully connected)
    x = Dense(100, activation='relu')(x)  # Classify the data into 100 f
    x = Dense(50, activation='relu')(x)  # Classify the data into 50 fea
    # categorical output of the angle
    angle_out = Dense(units=1, activation='linear', name='angle_out')(x)

    # continous output of throttle
    throttle_out = Dense(units=1, activation='linear', name='throttle_ou

    model = Model(inputs=[img_in], outputs=[angle_out, throttle_out])

    model.compile(optimizer='adam',
                  loss={'angle_out': 'mean_squared_error',
                        'throttle_out': 'mean_squared_error'},
                  loss_weights={'angle_out': 0.5, 'throttle_out': .5})
```

Deep Learning - Part III

Keyfacts of our deep Network:

1. VGG16 Architecture (convolutional neural networks)
2. 266.628 Parameters
3. Based on Nvidias research
4. The final version has a Pooling Layer (basic implementation not)



Autonomously Driving

Keras Pilot - Part I

The Keras Pilot (*.h5* Format) is stored on the Raspberry Pi.

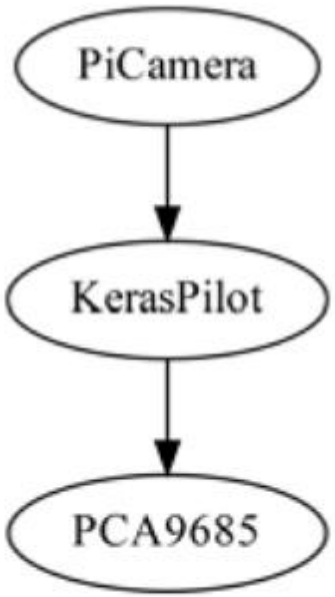
- The image captured with the PiCamera are 160x120x3 Pixels
- They pass through the follwing Neural Network
- The Output in the last fully connected Layer is (*angle_out* and *throttle_out*)

Convolutional Layer	Anzahl der Filter	Größe der Filter	Strides	Pooling	Aktivierungsfunktion
Convolution2D_1	24	5×5	2	Nein	Rectifier
Convolution2D_2	32	5×5	2	Nein	Rectifier
Convolution2D_3	64	5×5	2	Nein	Rectifier
Convolution2D_4	64	3×3	2	Nein	Rectifier
Convolution2D_5	64	3×3	1	Nein	Rectifier

Keras Pilot - Part II

Connection with the PiCamera Module:

- We use 20 frames per second (FPS)
- We use Tensorflow 1.1.17 for version compatibility
- According to that, the car changes 20 times per second the angle
- The throttle is fix, because of the space limitation



Evaluation

Evaluation - Part I

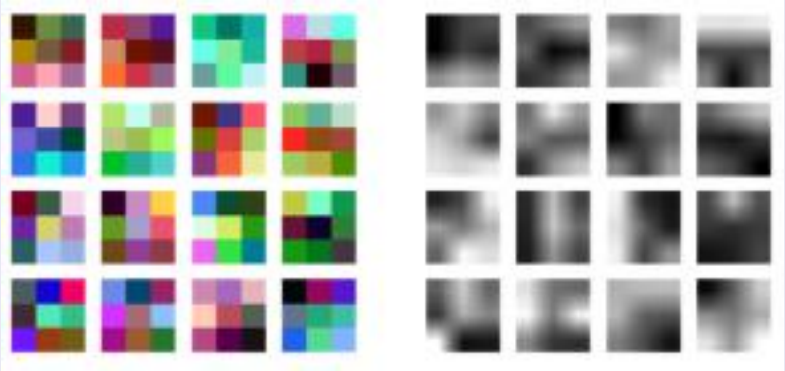
Evaluation

Even though CNN perform a really good result in the computer vision field, it is still quite hard to decompse the decisions and the filters to understand the blackbox behind. For this reason visualized one decision by plotting the occlusion maps (next page) based on this criterion function

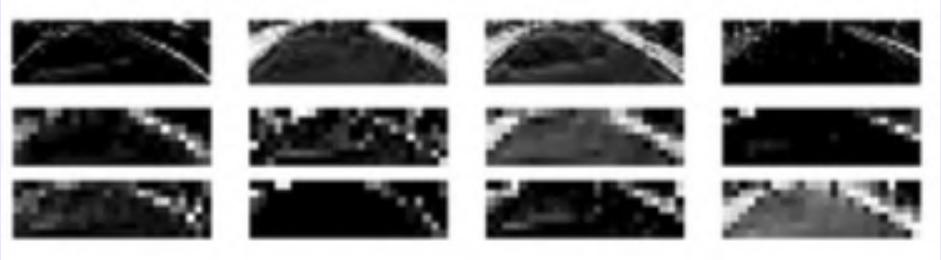
The criterion function:

$$O_{i,j} = \begin{cases} 0, & \text{if } abs(\hat{y}_{i,j} - y) > \epsilon \\ 1 & \text{otherwise} \end{cases} \tag{1}$$

Evaluation - Part II: Occlusion maps generated out of the CNN layers from section *Autonomously Driving - Keras Pilot (Part I)*



Not interpolated filter and gray with bilinear interpolation



Activation features after each Layer of the NN respectively

Conclusion

Final Thoughts

The project was challenging, but all of us learned a lot during this 2 semester long process of building up this project. While doing this project, Tensorflow 2.0 was released and this changed a lot. We needed to adjust constantly, because the Tech-research is so fast. In conclusion, we had a great time and are eager to dive deeper into Deep Learning!

Acknowledgement

The authors would like to thank Prof Dr. Florian Gallwitz from the University of Applied Science - Georg Simon OHM in Nuremberg.

References

- Github: https://github.com/bohniti/it-projekt
- Website: https://bohniti.github.io/it-projekt/project/

Sources

All of our sources are listed in the project paper on Github.

Thanks for listening!

The End

TECHNISCHE HOCHSCHULE NÜRNBERG
GEORG SIMON OHM